

# Vinci Unitree — Day 6 Lecture Notes: G1 Safe Locomotion, Arm Actions, and Integration

**Course:** Vinci Unitree Robotics Training

**Day:** Day 6

**Lecture length:** 3 hours

**Platform:** Unitree **G1** humanoid

**Stack:** Unitree SDK2 Python, CycloneDDS, high-level RPC clients, DDS topics, and supervised arm SDK streaming

**Audience:** Train-the-trainer cohort, robotics instructors, lab facilitators, and technical mentors

**Copyright:** © Vinci AI. Internal training material.

**Instructor framing.** Day 6 is not a choreography day. It is a safety, command-ownership, and integration day. Students learn how to move from read-only G1 readiness into carefully bounded humanoid locomotion and arm actions while preserving a clear distinction between **high-level RPC commands**, **DDS topic streaming**, and **out-of-scope full-body low-level motor control**.

## 1. Day 6 purpose and learning outcomes

Day 6 builds directly on Day 5. Day 5 established the G1 communication stack, the wired network interface, CycloneDDS readiness, `rt/lowstate` observation,

`MotionSwitcherClient.CheckMode()`, and the rule that **FSM damp is a motion blocker**.

Day 6 uses that foundation to introduce safe high-level locomotion, scripted upper-body actions, optional arm SDK streaming, and a final integration policy for choosing the correct control surface.

Unitree's G1 developer documentation describes G1 as a humanoid robot with upper-body and lower-body degrees of freedom, with the basic G1 commonly described as a 23-DOF platform and G1-EDU configurations varying with optional wrists, waist, and dexterous-hand equipment.<sup>1</sup>

Unitree's SDK2 Python repository describes SDK2 Python as the Python interface for Unitree SDK2 and shows both request-response service-style examples and DDS topic subscription/publishing examples.<sup>3</sup> These two facts define the Day 6 teaching problem: the same robot can be controlled through multiple software surfaces, so the instructor must teach students **which surface is appropriate for each job**.

| Outcome area                  | Students should be able to do this by the end of Day 6                                                                                                                       | Evidence to collect                                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Readiness gate</b>         | Re-run the Day 5 readiness gate before every motion lab and explain why <code>rt/lowstate</code> , <code>CheckMode == "ai"</code> , and FSM not equal to damp are mandatory. | Dry-run log showing <b>Readiness: PASS</b> .                           |
| <b>High-level locomotion</b>  | Use <code>LocoClient</code> commands such as <code>Move</code> , <code>StopMove</code> , <code>LowStand</code> , and <code>HighStand</code> under instructor supervision.    | Short motion log or observation note.                                  |
| <b>High-level arm actions</b> | Use <code>G1ArmActionClient</code> to execute named gestures such as <code>face wave</code> and <code>release arm</code> through the arm RPC service.                        | <code>--show-map</code> output and one successful supervised sequence. |
| <b>Arm SDK streaming</b>      | Explain how <code>rt/arm_sdk</code> differs from the high-level arm RPC path and why it is treated as a stricter safety class.                                               | Four-stage timing notes from the arm SDK run.                          |
| <b>Integration policy</b>     | Choose between locomotion, arm RPC, and arm SDK streaming for a classroom scenario without mixing command paths unsafely.                                                    | Completed decision guide and mixing-policy explanation.                |

## 2. Three-hour teaching plan

The official Day 6 repository sequence is longer than a single uninterrupted lecture, so the 3-hour session should be facilitated as an integrated lecture-lab. The instructor should prioritize readiness, one visible locomotion demonstration, one visible high-level arm action, a conceptual arm SDK explanation or supervised run, and a final integration discussion. If the room, robot condition, or time is constrained, the safest shortened path is: **Lab 0 readiness** → **Lab 1 or Lab 3 high-level gesture** → **Lab 2 locomotion short walk** → **Lab 5 comparison**.

| Time      | Segment                                              | Teaching emphasis                                                                                                  | Suggested evidence                             |
|-----------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| 0:00–0:15 | Opening, safety contract, Day 5 recap                | Why humanoid motion requires stricter staging than read-only DDS work.                                             | Students state the readiness gate aloud.       |
| 0:15–0:40 | Lab 0: readiness, DOF variant, control surfaces      | <code>G1ArmActionClient</code> , <code>rt/arm_sdk</code> , and <code>rt/lowcmd</code> are different authorities.   | Dry-run readiness log.                         |
| 0:40–1:10 | High-level locomotion lecture and dry run            | <code>LocoClient</code> service calls, velocity commands, and why <code>StopMove()</code> follows velocity only.   | Dry-run output and command map.                |
| 1:10–1:35 | Supervised locomotion demonstration                  | Short forward motion or stance demonstration with spotter and clearance.                                           | Motion log; optional video.                    |
| 1:35–1:45 | Break and robot settle                               | Check posture, battery, clearance, and network state.                                                              | Instructor confirms robot state.               |
| 1:45–2:15 | High-level arm action lecture and demo               | <code>G1ArmActionClient</code> , <code>action_map</code> , <code>face wave</code> , and <code>release arm</code> . | <code>--show-map</code> ; action sequence log. |
| 2:15–2:45 | Arm SDK streaming explanation or supervised run      | <code>rt/arm_sdk</code> , <code>LowCmd_</code> , 20 ms cadence, index-29 enable/disable, arm5 versus arm7.         | Stage timing notes or source walk-through.     |
| 2:45–3:00 | Integration policy, troubleshooting, knowledge check | One path per run, pause, readiness, neutral-first recovery.                                                        | Completed decision guide.                      |

**Trainer note.** The schedule should be slowed down if participants are unfamiliar with G1 body mechanics. It is better to complete one safe, well-explained demonstration than to rush through multiple impressive actions while students do not understand who owns the command stream.

### 3. Safety contract for Day 6

Humanoid operation introduces risks that are less intuitive than wheeled or quadruped demos. Arms sweep through human head and torso space. Standing balance depends on posture, floor friction, battery state, and current locomotion mode. A command that returns RPC code `0` means the request was accepted by the service; it does not prove that the physical motion is safe, complete, or visually obvious. The physical robot, not the terminal, is the final source of truth.

| Rule                              | Practical meaning                                                                                                                                                        | Instructor enforcement                                               |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Readiness before motion</b>    | Run the read-only readiness gate before locomotion, arm actions, or arm SDK streaming.                                                                                   | No PASS means no motion lab.                                         |
| <b>Clearance</b>                  | Maintain at least <b>2 m</b> around upper-body gestures and at least <b>3 m</b> forward clearance for walking.                                                           | Mark floor boundaries before sending commands.                       |
| <b>Spotter</b>                    | A spotter must observe the first run of every motion category.                                                                                                           | Assign one person to the robot, one to the keyboard, one to notes.   |
| <b>One command path at a time</b> | Do not interleave arm SDK streaming and arm RPC gestures in one uncontrolled process.                                                                                    | Stop, wait, re-check readiness, then switch.                         |
| <b>Stop after velocity</b>        | <code>StopMove()</code> is required after <code>Move(...)</code> , but it should not be blindly sent after an arm wave because it can cancel or interfere with gestures. | Teach the difference between velocity commands and gesture commands. |
| <b>Neutral-first recovery</b>     | Use <code>release arm</code> after many arm gestures or before an integration recap.                                                                                     | End demos with arms in a neutral high-level state.                   |
| <b>No low-level experiments</b>   | <code>rt/lowcmd</code> is a different safety class and is out of scope for Day 6.                                                                                        | Do not modify low-level motor examples in this lecture.              |

Unitree's quick-start documentation treats damping as an emergency or special state and warns that command conflicts can occur when control programs send instructions unexpectedly.[2](#)

Unitree's basic-services documentation also distinguishes low-level state topics from low-level command topics such as `rt/lowcmd`, which is why this course separates observation, high-level services, arm-only streaming, and full-body low-level authority.<sup>8</sup>

---

## 4. Day 6 architecture map

The Day 6 architecture can be taught as three layers. The first layer is **read-only observation** through `rt/lowstate`; this tells the class whether DDS messages are flowing. The second layer is **high-level service control**, where clients such as `LocoClient` and `G1ArmActionClient` ask onboard services to perform named or parameterized behaviors. The third layer is **streamed joint target control**, where the Day 6 arm SDK lab publishes `LowCmd_` messages to `rt/arm_sdk` for upper-body motion.

| Surface                        | Mechanism                                                       | Course use                                                                             | Main risk                                                                            |
|--------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>rt/lowstate</code>       | DDS subscription                                                | Read-only state observation; readiness check.                                          | Misreading absence of data as a robot fault when it may be network setup.            |
| <code>LocoClient</code>        | High-level sport RPC service                                    | Stance, short velocity motion, wave/handshake variants.                                | Forgetting <code>StopMove()</code> after velocity or using unsafe menu items.        |
| <code>G1ArmActionClient</code> | High-level arm RPC service named "arm"                          | Predefined gestures such as <code>face wave</code> and <code>release arm</code> .      | Running contact-style gestures near people or failing to release arms.               |
| <code>rt/arm_sdk</code>        | DDS publishing of <code>LowCmd_</code> at about 20 ms intervals | Custom upper-body interpolation using <code>arm5</code> or <code>arm7</code> examples. | Controller fighting, wrong DOF variant, unsafe gains, or overlapping command owners. |
| <code>rt/lowcmd</code>         | Full-body low-level command topic                               | <b>Out of scope</b> for Day 6.                                                         | Whole-body low-level motor authority; different safety class.                        |

The official SDK2 Python high-level G1 examples include locomotion and arm-action examples, including `g1_loco_client_example.py` , `g1_arm_action_example.py` , and the arm SDK streaming examples `g1_arm5_sdk_dds_example.py` and `g1_arm7_sdk_dds_example.py` .<sup>4</sup> Day 6 wraps and constrains those examples into classroom-safe scripts.

---

## 5. Lab 0 lecture: readiness, DOF variant, and control surfaces

Lab 0 is a non-motion readiness and orientation block. Its purpose is to stop students from treating all G1 APIs as equivalent. The instructor should explicitly compare `G1ArmActionClient` , `rt/arm_sdk` , and `rt/lowcmd` , then require students to rerun the Day 5 readiness gate before any Day 6 motion.

The readiness command should be run from the machine physically connected to the robot network, not from a cloud environment. Replace `en6` or `enp0s31f6` with the actual wired interface.

```
conda activate unitree_env
export CYCLONEDDS_HOME="$HOME/cyclonedds/install"
unset CYCLONEDDS_URI
cd /path/to/vinci-unitree

python course/day-05/lab-02/lab02_fsm_readonly.py en6
```

| Check                                         | Expected state                    | Why it matters                                     |
|-----------------------------------------------|-----------------------------------|----------------------------------------------------|
| <code>rt/lowstate</code>                      | Messages are flowing.             | DDS and network are alive.                         |
| <code>MotionSwitcherClient.CheckMode()</code> | <code>name == "ai"</code> .       | High-level behavior stack is in the expected mode. |
| FSM                                           | Not <code>1</code> / not damp.    | Damp blocks or makes high-level motion unsafe.     |
| DOF variant                                   | Instructor confirms arm5 or arm7. | Wrong arm SDK example may command invalid joints.  |

For upper-body streaming, the class must distinguish **23-DOF** and **29-DOF** configurations. In the upstream arm SDK examples, `g1_arm5_sdk_dds_example.py` is the usual starting point for 23-DOF units, while `g1_arm7_sdk_dds_example.py` adds wrist pitch and yaw chains for higher-DOF variants. The Day 6 course material also notes that joint index **29** is used in the arm SDK examples as an enable/disable field for the arm SDK stream. Students should learn this as a file-specific convention rather than as a number to reuse blindly.

## 6. Lab 1 and Lab 2 lecture: high-level locomotion with `LocoClient`

The locomotion portion teaches that G1 high-level movement is a service-mediated request to the onboard locomotion stack, not a raw motor command. Unitree's sport-services documentation describes the high-level pattern of initializing a sport client, setting a timeout, and sending speed, posture, or trajectory commands through named methods.<sup>7</sup> On G1, the Day 6 course uses `LocoClient` to stage short, supervised motions.

| Method                                             | Practical meaning in class                     | Teaching caution                                                                     |
|----------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>WaveHand()</code>                            | Loco-service wave gesture.                     | May be subtle or not visibly arm-like on some units.                                 |
| <code>HighStand()</code> / <code>LowStand()</code> | Requests a taller or shorter standing posture. | May return code <code>0</code> with little visible change.                           |
| <code>Move(vx, vy, vyaw)</code>                    | Sends a velocity command.                      | Requires open space and a following <code>StopMove()</code> .                        |
| <code>StopMove()</code>                            | Sends zero velocity.                           | Use after velocity; do not blindly send after arm gestures.                          |
| <code>Damp()</code>                                | Sends the robot toward damp behavior.          | Avoid in normal Day 6 labs unless intentionally recovering under instructor control. |

The default visible sequence in `lab03_loco_sequence.py` is deliberately conservative. It performs readiness, sends a visible arm `face wave` through `G1ArmActionClient` , waits, and then calls `release arm` . The optional `--loco-wave` flag compares the locomotion service's wave behavior, which may be less visually obvious. The script explicitly avoids calling `StopMove()` after a wave because the official locomotion example waits after `WaveHand()` rather than stopping velocity.

```
# Dry run: readiness only, no motion.
python course/day-06/lab-01/lab03_loco_sequence.py en6 --dry-run

# Default visible wave sequence.
python course/day-06/lab-01/lab03_loco_sequence.py en6

# Optional comparison: locomotion-service wave, may be subtle.
python course/day-06/lab-01/lab03_loco_sequence.py en6 --loco-wave
```

The separate `lab04_loco_motion.py` script focuses on velocity, stance, and balance. Its default forward movement is short and classroom-bounded. The script validates speed, waits for `rt/lowstate` , checks the motion mode, blocks damp, and always follows velocity segments with `StopMove()` .



```
# Dry run: readiness only.
python course/day-06/lab-02/lab04_loco_motion.py en6 --dry-run

# Short forward motion.
python course/day-06/lab-02/lab04_loco_motion.py en6

# Optional, instructor-approved variations.
python course/day-06/lab-02/lab04_loco_motion.py en6 --include-stance
python course/day-06/lab-02/lab04_loco_motion.py en6 --strafe
python course/day-06/lab-02/lab04_loco_motion.py en6 --yaw
```

| Scenario                                     | Recommended instructor response                                                                |
|----------------------------------------------|------------------------------------------------------------------------------------------------|
| RPC returns 0 , but nothing visible happens. | Check FSM, posture, floor contact, and whether the command was a subtle stance or wave action. |
| Student wants a longer walk.                 | Reduce speed first, mark a larger test area, and run one segment at a time.                    |
| Robot drifts after a velocity command.       | Send StopMove() again through the approved script or high-level menu.                          |
| Student tries to use damp as a normal stop.  | Reframe damp as an emergency or special state, not routine classroom flow.                     |

## 7. Lab 3 lecture: high-level arm actions with G1ArmActionClient

The arm-action portion introduces the high-level "arm" RPC surface. The class should understand that a named action such as face wave is translated by the SDK's action\_map into an integer ID, and ExecuteAction(action\_id) sends that ID to the robot. This is very different from streaming joint targets. In high-level arm action mode, the onboard action player owns the motion plan.

The official SDK2 Python arm-action example demonstrates the same pattern: initialize DDS, create G1ArmActionClient , set a timeout, initialize the client, and call ExecuteAction using the action map.[6](#)

| Action name                                 | Typical classroom policy                                    |
|---------------------------------------------|-------------------------------------------------------------|
| face wave                                   | Default safe visible gesture.                               |
| high wave                                   | Possible approved variation with clearance.                 |
| release arm                                 | Neutral/release habit after gestures.                       |
| shake hand                                  | Instructor-approved only; person-contact style.             |
| high five                                   | Instructor-approved only; person-contact style.             |
| hug                                         | Avoid in normal classroom demonstrations.                   |
| hands up , heart , clap ,<br>reject , x-ray | Use only after checking clearance and instructor allowlist. |

Students should first print the static map without robot motion, then run a dry run, and only then perform the default sequence.

```
# No robot required if SDK imports correctly.
python course/day-06/lab-03/lab01_arm_action_sequence.py --show-map

# Robot-side dry run.
python course/day-06/lab-03/lab01_arm_action_sequence.py en6 --dry-run

# Optional live firmware list, when supported.
python course/day-06/lab-03/lab01_arm_action_sequence.py en6 --list-action

# Default action: face wave, then release arm.
python course/day-06/lab-03/lab01_arm_action_sequence.py en6
```

**Important distinction.** `GetActionList()` may be unavailable or may return firmware-specific names. The course script uses the SDK `action_map` names. The numeric ID is the actual payload used by `ExecuteAction`, so students should not assume that every displayed text label is interchangeable across firmware builds.

## 8. Lab 4 lecture: arm SDK streaming through `rt/arm_sdk`

The arm SDK lab is the conceptual center of Day 6 because it changes the student's mental model. In the high-level arm RPC lab, students requested a named behavior and the robot planned the motion. In the arm SDK lab, the script publishes `LowCmd_` messages on `rt/arm_sdk` about every **20 ms**. The script therefore owns interpolation, target positions, gains, and enable/disable timing. That is why this lab requires a supervisor, a readiness gate, correct DOF selection, and no overlapping arm-control process.

| Comparison point      | High-level arm RPC                                | <code>rt/arm_sdk</code> streaming                                             |
|-----------------------|---------------------------------------------------|-------------------------------------------------------------------------------|
| Communication         | Request-response RPC service <code>"arm"</code> . | DDS publish topic <code>rt/arm_sdk</code> .                                   |
| Payload               | Integer action ID in an RPC body.                 | <code>LowCmd_</code> with joint targets, gains, and CRC.                      |
| Motion planning       | Onboard action player.                            | Course or vendor script interpolates.                                         |
| Typical classroom use | Quick predefined gestures and release.            | Demonstrating joint-stream control concepts.                                  |
| Safety concern        | Gesture sweep volume and person contact.          | Wrong DOF variant, gains, conflicting publishers, or enable/disable mistakes. |

The course wrapper `lab02_run_arm_sdk.py` adds safeguards around the vendor examples. It runs the same readiness gate, validates the selected variant, rejects unsupported vendor execution on macOS, asks for explicit confirmation unless `--yes` is supplied, and by default launches the portable course implementation.

```
# Dry run: readiness only.
python course/day-06/lab-04/lab02_run_arm_sdk.py en6 --dry-run

# 23-DOF style arm5 path, typical classroom default.
python course/day-06/lab-04/lab02_run_arm_sdk.py en6 --variant arm5

# 29-DOF style arm7 path, only when confirmed by instructor.
python course/day-06/lab-04/lab02_run_arm_sdk.py en6 --variant arm7
```

The four-stage streaming sequence is the safest way to explain what the script is doing internally.

| Stage | Approximate time window with<br><code>duration_ = 3.0</code><br>s | What the stream does                                                                   | Teaching interpretation                                  |
|-------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------|
| 1     | 0–3 s                                                             | Enables arm SDK using the special index-29 enable field and blends toward a zero pose. | The script claims upper-body command authority.          |
| 2     | 3–9 s                                                             | Interpolates toward the target arm pose.                                               | Students observe the main visible arm lift.              |
| 3     | 9–18 s                                                            | Blends back toward the measured or neutral pose.                                       | The script exits the demonstration trajectory gradually. |
| 4     | 18–21 s                                                           | Ramps the arm SDK enable field back toward disable.                                    | Command ownership is returned cleanly.                   |

The course implementation publishes `LowCmd_` with CRC and uses separate joint lists and target poses for `arm5` and `arm7`. It assigns `kp` and `kd` values in the script and reads feedback from `rt/lowstate` to blend from measured joint positions. Students should not edit gains casually. If the instructor permits a parameter change, the safest educational change is to increase the duration in a copy of the file, which slows the interpolation without changing stiffness.

## 9. Lab 5 lecture: comparison and integration policy

The final integration lab is intentionally conservative. It does not attempt a complex whole-body performance. Instead, it prints a decision guide and runs an RPC-only recap: `release arm` → `face wave` → `release arm`. This teaches the class that integration means **choosing and sequencing command paths**, not simply mixing all APIs in one script.

```
# No robot: print comparison guide.
python course/day-06/lab-05/lab03_integrate.py --compare-only

# Robot-side readiness after previous labs.
python course/day-06/lab-05/lab03_integrate.py en6 --dry-run

# Default recap: release arm, face wave, release arm.
python course/day-06/lab-05/lab03_integrate.py en6
```

| Decision question                                       | Recommended path                                                            |
|---------------------------------------------------------|-----------------------------------------------------------------------------|
| Need a named gesture the robot already knows?           | <code>G1ArmActionClient</code> through the <code>"arm"</code> RPC service.  |
| Need a short forward or rotational body movement?       | <code>LocoClient</code> , with <code>StopMove()</code> after velocity.      |
| Need to shape arm joint positions over several seconds? | <code>rt/arm_sdk</code> , supervised, with correct DOF variant.             |
| Need full-body low-level motor control?                 | Out of scope for Day 6. Escalate to an advanced safety review.              |
| Unsure which path owns the robot right now?             | Stop, wait, run readiness, and use <code>release arm</code> if appropriate. |

The classroom mixing policy should be repeated at the end of Day 6. Students should finish one pipeline completely, wait at least 10 seconds before switching, re-check readiness, and avoid interleaving arm SDK streaming with RPC gestures in the same process. If a motion looks wrong, the correct response is not to lower gains or send more commands. The correct response is to stop scripts, return to read-only observation, recover posture, and ask the instructor.

## 10. Instructor demonstration script

A safe instructor demonstration can be conducted as the following staged sequence. The instructor should announce each step before running it and should require students to predict the expected physical outcome.

| Step | Instructor command or action                          | Expected observation                                         | Teaching question                                               |
|------|-------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------|
| 1    | Run Day 5 readiness check.                            | PASS with lowstate, AI mode, and non-damp FSM.               | Why is read-only state checked before motion?                   |
| 2    | Run locomotion dry run.                               | No physical motion.                                          | What did the script prove and what did it not prove?            |
| 3    | Run default high-level wave sequence.                 | Visible wave, then release arm.                              | Which API produced the visible wave?                            |
| 4    | Run short <code>Move</code> segment if space permits. | Robot steps forward briefly, then stops.                     | Why is <code>StopMove()</code> mandatory here?                  |
| 5    | Print arm action map.                                 | Static action names and IDs.                                 | Why are names not the same as physical safety approval?         |
| 6    | Explain or run arm SDK stream.                        | Four-stage arm motion, then <code>Done!</code> .             | Who owns interpolation in this lab?                             |
| 7    | Run integration recap.                                | <code>release arm →<br/>face wave →<br/>release arm .</code> | Why does the recap avoid mixing arm SDK and RPC in one process? |

## 11. Troubleshooting matrix

Troubleshooting on Day 6 should start from the bottom of the stack and move upward. Do not diagnose a gesture problem before confirming network interface, CycloneDDS environment, `rt/lowstate` , motion mode, and FSM state.

| Symptom                                                     | Likely cause                                                                  | Instructor response                                                                          |
|-------------------------------------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| No <code>rt/lowstate</code> messages.                       | Wrong interface, static IP issue, robot not fully booted, or DDS setup issue. | Verify wired interface, ping robot, unset <code>CYCLONEDDS_URI</code> , and rerun readiness. |
| <code>CheckMode</code> is not <code>ai</code> .             | Robot is in another mode or command ownership is not as expected.             | Follow the field guide and do not begin motion.                                              |
| FSM is <code>1</code> / damp.                               | Robot is in a damping or blocked state.                                       | Recover posture and rerun Day 5 readiness; do not send Day 6 motion.                         |
| RPC returns <code>0</code> , but motion is subtle.          | Stance and some wave commands may be visually small on certain firmware.      | Compare with the default arm RPC wave; confirm physical state.                               |
| Robot moves after <code>Move</code> but continues drifting. | Velocity command not stopped or stop was missed.                              | Send <code>StopMove()</code> through approved high-level path.                               |
| Arm action name is rejected.                                | Name does not exactly match SDK <code>action_map</code> .                     | Run <code>--show-map</code> ; copy names exactly.                                            |
| <code>GetActionList()</code> fails.                         | Firmware may not support the list call.                                       | Use static <code>action_map</code> and document the limitation.                              |
| Arm SDK does not move.                                      | Wrong variant, enable not active, no publisher, or readiness fault.           | Confirm arm5/arm7, run dry-run, inspect stream stage output.                                 |
| Arm SDK motion is jerky or appears to fight.                | Conflicting control path, unsafe gains, unstable posture, or wrong variant.   | Abort, stop scripts, run readiness, and do not edit gains without instructor approval.       |

| Symptom                      | Likely cause                                          | Instructor response                                                              |
|------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------|
| Arms remain high after demo. | Action or stream ended without a clean neutral state. | Try approved <code>release arm</code> once, then re-check readiness and posture. |

## 12. Assessment and knowledge checks

The assessment should test safety reasoning as much as command recall. A student who can list ten commands but cannot explain command ownership is not ready to facilitate G1 motion labs.

| Question                                                               | Expected answer                                                                                                                   |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| What three conditions define the Day 6 readiness gate?                 | <code>rt/lowstate</code> flowing, <code>CheckMode = "ai"</code> , and FSM not equal to damp.                                      |
| When is <code>StopMove()</code> required?                              | After velocity commands such as <code>Move(vx, vy, vyaw)</code> .                                                                 |
| Why should <code>StopMove()</code> not be blindly called after a wave? | It sends zero velocity and can interfere with or cancel gesture behavior.                                                         |
| What service does <code>G1ArmActionClient</code> use?                  | The high-level arm RPC service named <code>"arm"</code> .                                                                         |
| What topic does the arm SDK lab publish to?                            | <code>rt/arm_sdk</code> .                                                                                                         |
| Why does the arm SDK lab require arm5 versus arm7 selection?           | Different G1 DOF variants have different available wrist joints.                                                                  |
| What is the safe integration policy after an arm SDK run?              | Finish the run, wait, re-check readiness, use neutral-first recovery, and do not interleave streams and RPC calls in one process. |
| Why is <code>rt/lowcmd</code> excluded from Day 6?                     | It is full-body low-level motor authority and belongs to a different safety class.                                                |



## Short written exercise

Ask students to write a five-sentence decision guide for a visitor demonstration. Their answer should include the readiness gate, the chosen API path, the physical clearance requirement, how the action stops or returns to neutral, and what evidence they will record.

## Practical instructor sign-off

A participant is ready to assist with Day 6 facilitation only when they can show the following evidence.

| Required artifact   | Acceptance criterion                                                                        |
|---------------------|---------------------------------------------------------------------------------------------|
| Readiness log       | Shows PASS immediately before motion.                                                       |
| Locomotion note     | Identifies when <code>StopMove()</code> is used and why.                                    |
| Arm action map      | Shows action names and IDs from <code>--show-map</code> .                                   |
| Arm SDK explanation | Correctly describes <code>rt/arm_sdk</code> , <code>arm5/arm7</code> , and the four stages. |
| Integration policy  | States one path per run, pause, re-check, and neutral-first recovery.                       |

## 13. Day 7 readiness bridge

Day 6 prepares students for Day 7 by teaching that each subsystem needs a clear owner. Day 7 introduces audio, speech, hand/peripheral behavior, or capstone work depending on the course track. Those activities still depend on the same discipline: establish communication, observe state, choose one control surface, run a bounded command, collect evidence, and stop cleanly.

The most important Day 6 habit to carry forward is **state before action**. Every future G1 feature should be introduced by asking: What state can we observe? Which client or topic owns the subsystem? What is the stop or release behavior? What evidence proves that the robot remained safe and recoverable?

## 14. References